Using a Symbolic Knowledge Graph to Address LLM Limitations in Analog Circuit Topology Generation

Hashmath Shaik
Department of Electrical and Computer Engineering
Stony Brook University
Stony Brook, NY
hashmath.shaik@stonybrook.edu

Alex Doboli
Department of Electrical and Computer Engineering
Stony Brook University
Stony Brook, NY
alex.doboli@stonybrook.edu

2025 IEEE 15th Annual Computing and Communication Workshop and Conference (CCWC) | 979-8-3315-0769-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/CCWC62904.2025.10903951

***Abstract*—Analog circuit topology synthesis has been shown to be hard arguably due to the difficulty to formulate it either as a regression or as an optimization problem, which are currently the two main synthesis approaches. This paper proposes a novel analog circuit topology synthesis methodology that combines the capabilities of Deep Neural Models with the advantages of symbolic representations. It integrates Large Language Models (LLMs) with Symbolic Knowledge Graphs (SKGs): an LLM offers broad descriptions of the topological modules that can be incorporated into the final schematics, and an SKG incorporates all design details needed to complete the topological modules and other related circuitry of a circuit schematics. The SKG also supports the causal reasoning flow through which the modules indicated by the LLM are selected and integrated with each other.**

***Index Terms*—LLM, symbolic knowledge graphs, topology generation, analog circuits.**

I. Introduction

Machine Learning (ML) has been already transforming many important domains, like education, healthcare, entertainment, governance, and so on [1]. ML algorithms pertain to two categories. Most methods attempt to represent (model) large quantities of data to recognize trends and distinguishing features used then in prediction, clustering, and classification. For example, many classifiers were trained on image data to recognize various diseases. The second category intends not only to create a representation of the training data but also to generate new outputs for new needs, e.g., images, text, audio data, etc. These methods, currently fewer than those in the first category, are called Generative ML (GML) [2].

Generative methods have been developed outside ML, such as the synthesis algorithms in Electronic Design Automation (EDA) to create optimized electronic circuits and systems [3]–[5]. Synthesis algorithms take as input a description of the design requirements and automatically convert it into a circuit design. Creating a description of the design as a starting point is an essential aspect of EDA synthesis. GML [2], i.e. Generative Adversarial Networks, transformers, Large Language Models (LLMs), and so on, eliminates the need for an explicit design description, instead it produces an internal, implicit description based on the training data. The description is then used to create a new output. Conceptually, new output generation requires two broad types of operations: *generalization* to produce the description for given problem requirements and *detailing* to transform the solution into the corresponding output. New electronic circuit generation in contrast to traditional GML tasks, like creating text, images, and sound, requires that the circuit offers optimized performance, hence its topology (structure) is justified based on the goal to achieve optimality. Hence, the circuits produced by GML must comprise of modules (sub-circuits) that can be causally explained by achieving performance optimality.

Existing work on using GLM, e.g., LLMs, in EDA tackles activities, like code generation, benchmarking, and debugging for Hardware Description Languages [6], improving EDA tool interoperability by generating executable scripts using Natural Language Processing [7], and supporting design tasks, e.g., AI accelerator design, layout design, and analog circuit design [8]. LLMs have been proposed to generate the topologies (schematics) for power converters [9]. The LLM must be pretrained with a set of converter circuits. The LLM generates new circuits by completing a circuit that some of its parts removed. Based on GPT-4, LADAC replicates a traditional design methodology, e.g., topology selection, mapping the requirements to the topology, and transistor sizing [10]. An LLM is proposed to summarize design documents and extracts design knowledge into a predefined template [11]. While this work presents pioneering ideas, the methods have limited ability to create topologies justified by their operation optimality.

This paper presents an analog circuit topology generation method that integrates an LLM with a Symbolic Knowledge Graph (SKG) to create optimized schematics for a set of problem requirements. The paper argues that while LLMs provide generalization capabilities, and hence can indicate the topological modules that are candidates to be part of a schematics, they are limited in elaborating the modules at the detailed MOSFET device-level. Moreover, they cannot introduce any module adjustments depending on concrete quantitative performance values, including tradeoffs and bot-

979-8-3315-0769-5/25/$31.00 ©2025 IEEE

Authorized licensed use limited to: University of Florida. Downloaded on September 28,2026 at 03:52:25 UTC from IEEE Xplore. Restrictions apply.

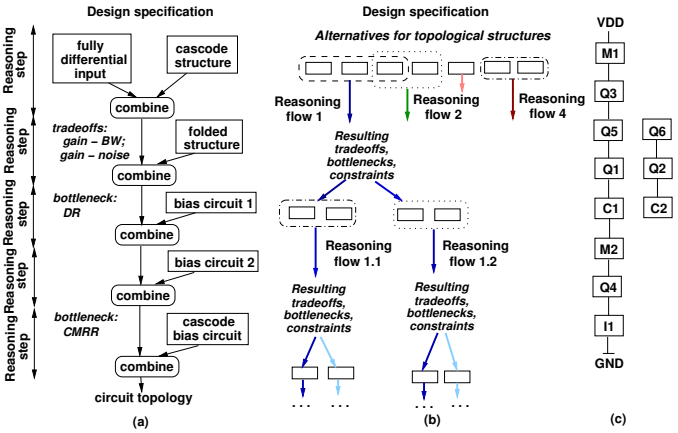


Fig. 1. (a) Performance-based justification of a circuit topology, (b) reasoning-based circuit design flow, and (c) LLM limitations for topology generation.

tlenecks, two main design elements. An SKG is proposed to compensate for these limitations. Experiments give evidence for the stated LLM limitations, which can then serve as starting points for developing the SKG-related features and algorithms.

The paper has the following structure. Section II motivates the presented work. Section III details the synthesis method. Experiments are in Section IV. Conclusions end the paper.

II. Motivation

The analog circuit topology design process can be summarized as a sequence of causal reasoning steps, in which every step decides the using and/or adaptation of a topological module to address the design specifications, e.g., design requirements, constraints, and bottlenecks [12]. For example, the first reasoning step might decide to use a fully differential input together with a cascode structure to achieve a high gain and high speed operation. The resulting structure has gain - bandwidth and gain - noise tradeoffs as well as a bottleneck in its limited dynamic range (DR). The next reasoning step includes specific biasing circuits into the circuit topology, which might produce new bottlenecks, like a limited common-mode rejection ratio (CMRR). Fig. 1(a) shows a sequence of reasoning steps to design a low-voltage, high-speed operational transconductor (OTA) [13].

Fig. 1(b) illustrates the reasoning flow that creates a circuit design. Multiple reasoning flows are possible for a design specification, each flow comprising of a sequence of reasoning steps. Each reasoning step uses the design specification to identify alternative topological structures, where structures are basic circuits, like complementary differential inputs, cascode structures, folded inputs, level shifting, biasing circuits, and so on. For example in Fig. 1(b), four reasoning flows originate depending on the clusters of selected structures, each structure being shown as a small rectangle and a cluster being marked with different lines. Selecting a cluster produces design tradeoffs between the parameters of the selected structures, constraints, and bottlenecks. The flow continues with reasoning steps that use the tradeoffs, constraints, and bottlenecks of the current structure to select the next topological module to be

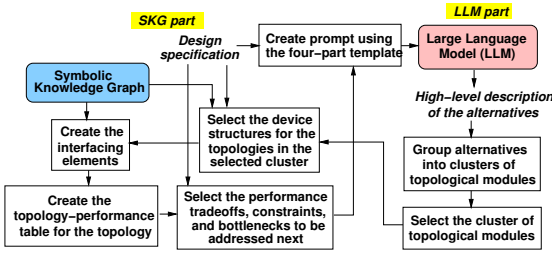


Fig. 2. Analog circuit synthesis using LLM - SKG.

added to the circuit topology. Hence, a reasoning step of a design flow is expressed as follows:

$$< DesReq, \{TopolStr_i\}, \\ < \{Tradeoff_j\}, \{Con_k\}, \{Bot_p\} >> \Rightarrow TopolStr_{new} \quad (1)$$

Parameter $DesReq$ indicates the design requirements, $\{TopolStr_i\}$ is the set of topological structures (modules) already included into the circuit design, the triplet $< \{Tradeoff_j\}, \{Con_k\}, \{Bot_p\} >$ are the set of tradeoffs, constraints, and bottlenecks of $\{TopolStr_i\}$, and $TopolStr_{new}$ is the topological structure that tackles the previous triplet by being added to the design.

Current LLMs have a limited capability to correctly generate the schematics of analog circuits.

Example: Microsoft's Copilot LLM was prompted with the following request "`Describe the schematics of class AB operational transconductor with floating capacitors`", and it created the following response: "`differential stage pair: Q1, Q2; cascode transistors: Q3, Q4 to increase the output impedance and gain; floating capacitors: C1, C2 float between different nodes; current mirrors: M1, M2 to give balanced current distribution; biasing circuit to dynamically adjust the biasing current to maintain class AB operation`". In addition, Fig. 1(c) was produced to show how the enumerated blocks connect to each other. Other reframings of the prompt generated similar answers. Additional prompts to get more details about the schematics offered some details, like the prompt "`What are Q5 and Q6 in the schematics of class AB operational transconductor`" produced three aspects in the LLM response, e.g., Q5 and Q6 are cascode transistors, auxiliary transistors to share the load with Q1 and Q2 to improve current handling, and part of the adaptive biasing.

Note that while the LLM response offers a broad description of a class AB OTA schematics and operation, the response is incomplete and only partially correct. For example, devices Q5 and Q6 of the topology in Fig. 1 were not mentioned in the description. Also, components Q6, Q2, and C2 were not connected to the rest of the circuit.

III. Synthesis using Large Language Models - Symbolic Knowledge Graphs

A. Synthesis Methodology

As shown in Fig. 1(b), the analog circuit synthesis methodology must generate multiple reasoning flows, including causal reasoning steps motivated by the triplets $< \{Tradeoff_j\}, \{Con_k\}, \{Bot_p\} >$ specific to each step.

Authorized licensed use limited to: University of Florida. Downloaded on September 28,2026 at 03:52:25 UTC from IEEE Xplore. Restrictions apply.

A reasoning flow includes two types of operations that must be replicated by the synthesis methodology: generalization and elaboration. Conceptually, generalization anticipates new design points located between two samples or predicts new points based on the trends defined by previous solutions. Regression is traditionally used for generalization [2]. Elaboration is the activity of adding details to an existing solution, e.g., the interconnected devices to implement a certain signal processing operation, like amplification. Elaboration is a top-down activity that has been usually tackled through transformation rules using a library of building blocks, like the basic circuit libraries used by the transformation rules of heuristics [4] or Genetic Algorithm-based analog circuit synthesis methods [5]. This paper argues that while LLMs are well suited for generalization they are limited for elaboration. Instead, the proposed methodology uses a Symbolic Knowledge Graph (SKG) to elaborate in conjunction with an LLM used to generalize.

To justify the proposed integration of an LLM and an SKG, we started from the mathematical model of an LLM, as expressed by equations (2) and (3) [2]:

$$Z = LNorm[ConCat[Softmax[\frac{XW_h^{(q)}XW_h^{(k)}}{\sqrt{D}}]XW_h^{(v)}]_h W^{(o)} + X] \quad (2)$$

$LNorm$ indicates layer normalization and $ConCat$ is the concatenation of the multiple attention heads. Matrix X is the input, where each row is an embedding of a input component. Matrices $W_h^{(q)}$, $W_h^{(k)}$, $W_h^{(v)}$, and $W^{(o)}$ are the queue, key, value, and output matrices [2]. The output is then:

$$\tilde{X} = LNorm[MLP[Z] + Z] \quad (3)$$

where MLP represents a multi-layer perceptron.

Equation (2) can be rewritten in a functional form to illustrate the cascaded nature of processing the input X:

$$Z = Gen[Rep[Assoc[X]]] \mid Context \quad (4)$$

Function $Assoc$ finds the associations between the input components. Function Rep maps its inputs to another space, e.g., the feature space. Function Gen produces the output. $Context$ is the context of performing specific computations.

In an LLM-only synthesis methodology, the reasoning steps of the flows shown in Fig. 1(b) should be implemented as expressed in equation (1) by using an LLM model described by equations (2)-(3) (or equation (4)). The next lemma indicates the difficulty of creating such an approach.

Lemma: Equations (2)-(3) (or equation (4)) are ineffective in modeling circuit topology elaboration.

Proof: Circuit topology elaboration requires adding new topological structures, e.g., parameter $TopolStr_{new}$ in equation (1). Each new topological structure includes a set of interconnected devices (e.g., MOSFETs) that tackle the triplet $< \{Tradeoff_j\}, \{Con_k\}, \{Bot_p\} >$ of specific tradeoffs, constraints, and bottlenecks. The composition $Rep[Assoc[X]]]$ is a feature representation of the design requirements and their associations. Hence, it would be function Gen that elaborates the schematics by introducing new interconnected devices. However, function Gen in an LLM is trained to offer regression among the samples of structural motifs used in training, which indicates that the new topological structure cannot go beyond the sample features. Moreover, training requires using an exponential number of samples to capture the design space due to the fact that all relationships between device connections and performance attributes must be learned.

TABLE I
TOPOLOGY - PERFORMANCE TABLE

Parameters	DC gain	CM gain	Noise	DP0	GPP0
g_{md3}	↓	↓	↑	↑	↑
g_{ms3}, g_{mg1}	↑	↑	↓	-	↑
$C_{gd3} + C_{db3}$	-	-	-	↓	↓
g_{ms1}	-	↓	↓	-	-
g_{mg5}, g_{ms5}, g_{mg3}	-	-	↓	-	-
$C_{gd1} + C_{gs1}$	-	-	↑	-	-
g_{md5}, g_{md1}	↓	↓	↑	-	↑

Hence, a circuit topology synthesis methodology must use an additional mechanism for topology elaboration in addition to using an LLM to identify the general topological modules used in an analog circuit. This paper suggests using an SKG to implement the reasoning flow used in elaboration.

Fig. 2 describes the proposed LLM - SKG-based circuit synthesis method. The design specification is used to create a prompt that is applied to the Large Language Model to obtain high-level descriptions of the topological modules that can be used to address the specification. The received alternatives are then clustered together into modules that should be used together in a schematics (see the top part of Fig. 1(b)). Then, the clusters are explored in the order of their likelihood to be part of the solution. Note the output of this step is a textual description of the selected topological modules, their parameters, benefits, and limitations. The SKG is then searched to select the device structures for the selected topological modules. Fig. 3(c) depicts a fragment of an SFG, and more details are offered in Subsection IV.C. The next step creates the interfacing elements between the topological modules. Subsection IV.C discusses this step. Next, the methodology generates the topology-performance table (TPT) for the created topology. TPT is then utilized to identify the performance, constraints, and bottlenecks of the topology, which then serve as input to create the next LLM prompt.

Table I is a TPT sample. It captures the impact of the circuit parameters on performance [12], i.e. the increasing (↑), decreasing (↓), or independence ($-$) of a performance metric (TPT columns) with regard to the circuit parameters (TPT rows). The table is used to select the performance bottlenecks to continue to refine a topology, e.g., prompting the LLM to suggest topological modules to enhance DC gain and CM gain.

B. Prompt Generation

Prompt generation must consider how an LLM was trained. A typical circuit design description, e.g., in a research paper or technical report, includes design requirements, background information, related work including a critique of other designs, a broad description of the proposed solution, a detailed description of the solution, and experimental results [14]. The

Authorized licensed use limited to: University of Florida. Downloaded on September 28,2026 at 03:52:25 UTC from IEEE Xplore. Restrictions apply.

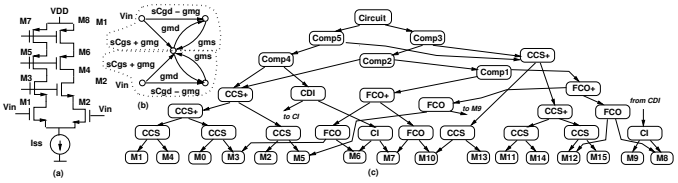


Fig. 3. (a) Circuit schematics, (b) frequency domain signal-flow graph, and (c) SKG fragment.

nature of the textual information used in training determines the characteristics of the learned matrices in equations (2)-(3).

In terms of the two equations, the background information offers the associations between applications, broad performance needs and constraints, which split the design space into sub-regions. For example, such statements used in LLM training indicate that switched-capacitor circuits are used in high-speed, low-voltage $\Delta\Sigma$ modulators operating at 2.5V [13].

Related work presents the topological features of similar designs, hence pertaining to the same design space sub-region as the current problem requirements, including the resulting performance improvements and limitations. For example, related work might indicate that the cascode topology improves the gain and high-speed operation of low supply voltage OTAs but limits their DR [13]. Or that a Miller two-stage OTA has a lower gain and limited bandwidth due to higher-order poles and poor PSRR [13]. According to equation (1), the related work data offers information on performance tradeoffs, constraints, and bottlenecks of various topological structures. However, they do not present the complete flow that produced a circuit, i.e. the complete sequence of reasoning steps, including the performance justification of each step.

The broad description of the solution enumerates topological similarities with previous designs as well as the changes in the topology and the produced performance improvements. For example, using a fully differential topology together with cascode and common-mode feedback structures improves CMRR [13]. The need to improve CMRR serves as a causal justification for using the two additional structures. The detailed description offers data on how the parameters of the newly added topology structure changes the performance triplet in equation (1). For example, setting the drain-source saturation voltages to be small sets a constraint for the circuit [13]. While the detailed description discusses multiple reasoning steps, the design flow might not be complete or correspond to a sequence of causal implications.

Lemma: An LLM prompt should include four parts: (i) Motivation, (ii) Goal, (iii) What should be included, and (iv) What should be avoided. Motivation offers the performance context of the prompt, e.g., tradeoffs, constraints, and bottlenecks. Goal describes the prompted information. The third part enumerates any topology structures that should be used together with the prompted answer, and the last part enumerates the topological structures to be excluded from the answer.

Proof: Even though only some of the four parts can occur in prompt, as previously explained, the above prompt structure likely matches the nature of the data used in the LLM training.

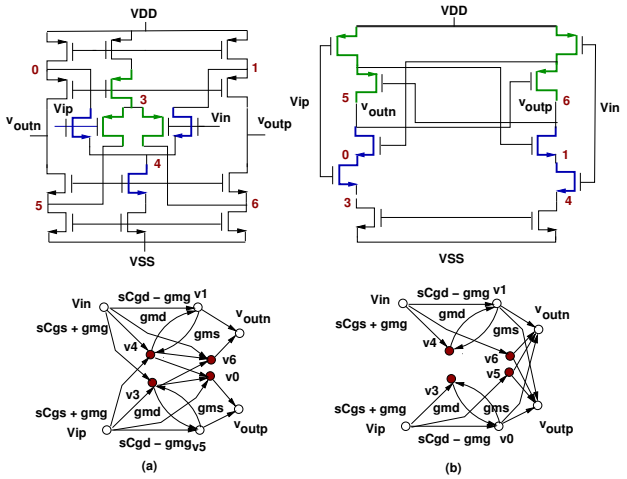


Fig. 4. Elaboration for local performance improvement: changing node connections in two similar circuit topologies and their associated SFGs.

C. Using Symbolic Knowledge Graphs in Design Elaboration

As explained in Section III, current LLMs arguably face difficulties in elaborating the topological features that are returned in response to the prompts about finding the circuit schematics that can implement the required design specification. These difficulties encompass three related issues: (i) finding the correct transistor structure corresponding to a topology indicated in an LLM's textual response, (ii) combining the transistor structures of the topologies that must be used together, and (iii) including any "glue" devices needed to guarantee to correct operation or to improve the performance of the combined modules. SKG is used with dedicated algorithms to tackle the three issues.

SKG stores the transistor structures of the topology modules. Fig. 3 depicts the used SKG. It clusters circuit topologies based on their similar structures, as shown in Fig. 3(c). The current SKG includes thirty four basic topologies, i.e. folded cascode (FCO), complementary inputs (CI), cascode current sources (CCS), and complementary differential inputs(CDI). The sign'+' indicates that multiple such structures can occur in parallel. CMOS devices are labeled as M_i. Each basic circuit has associated a parameterized schematics, like in Fig. 3(a) and a frequency-domain symbolic signal flow graph (SFG), as shown in Fig. 3(b). The SFGs of the selected basic circuits form the symbolic relations used for causal reasoning.

Generating the device structures needed to combine and guarantee the operation of the topological modules selected from the SKG, e.g., biasing circuits, must tackle three cases: Case A: improve the local performance of a topology module. e.g., pole-zero cancelation or gain boosting, Case B: providing adaptation to changing conditions over time, i.e. adaptive biasing, and Case C: offering global adjustment (control) based on signals observed throughout the entire circuit, including the output, such as global feedback structures.

Case A: Fig. 4 illustrates the elaboration of device structures to improve local performance. It shows two complementary input cascode topologies [13], which even though structurally different can be related to each other by their similar SFGs,

Authorized licensed use limited to: University of Florida. Downloaded on September 28,2026 at 03:52:25 UTC from IEEE Xplore. Restrictions apply.

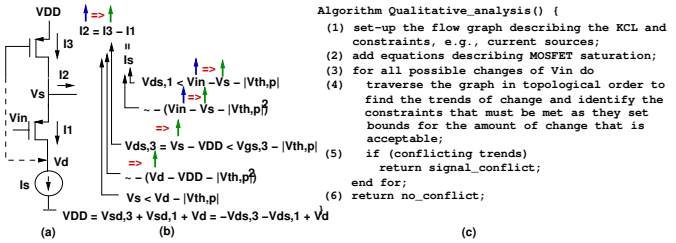


Fig. 5. Elaboration for adaptation over time: (a) adaptive biasing circuit, (b) causal requirements, and (c) qualitative simulation algorithm.

as shown at the bottom of Figs. 4(a) and (b). Their frequency behavior is defined by the two SFGs, which have the same number of nodes and also some similar edges. The main difference between the two is due to the connections between inputs, outputs, and nodes v_0, v_3, v_4 and v_6 (highlighted in red). Elaboration in this case refers to the connections of the latter four nodes.

The algorithm explores possible arcs between inputs, outputs, and internal nodes. This step changes the performance as it creates different expressions among the related parameters. New nodes are added to the SFG by replicating existing graph sub-structures, which represents having multiple, similar device structures, such as the pair of NMOS and PMOS input MOSFETs in the schematics of Fig. 4(a).

The analysis finds constraints for the device parameters, so that the local performance of a topology module is improved. For example, constraints might increase the expected frequency performance of the circuit by tackling its poles, e.g., through pole - zero cancelation or pushing the dominant poles to high frequencies.

Case B: The step considers the elaborations required to adapt to changing conditions over time. The proposed algorithm runs a qualitative simulation of the description to identify if the explored structure can adjust to the signal trends or if there are any built-in contradictions that prevent the adjustment. For example, checking the signal trends computes if the signals grow or decrease for certain changes of the operation conditions, the strengths of the trends (e.g., their maximum gradients), the number and kind of involved parameters, and the involvement of any performance tradeoffs.

Fig. 5 illustrates the elaboration step for this case. It shows in Fig. 5(a) an adaptive biasing circuit, in which detailing must decide the gate voltage of the top MOSFET device, so that current I_2 increases as a result of a higher input voltage V_{in}. The constraint that must be met by the circuit is that current I_2 increases ($\uparrow$) for a higher voltage V_in ($\uparrow$). The causal requirements imposed by the adaptation scheme are shown as the expression $\uparrow \Rightarrow\uparrow$ in Fig. 5(b). Fig. 5(b) depicts the signal flow graph for qualitative circuit simulation. Fig. 5(c) presents the qualitative simulation algorithm. The graph models Kirchhoff's Current Law (KCL) set-up for the circuit nodes (Step 1). Moreover, for each current, the graph indicates its expression based on the MOSFET operation in saturation mode and the required conditions to keep a MOSFET in saturation (Step 2). All causal requirements are then found for the graph elements (shown as $\uparrow \Rightarrow\uparrow$ in the figure) to describe the signal trends in the circuit (Steps 3 and 4). Conflicting trends are found to signal that the structure cannot implement the adaptation requirements (Step 5).

Case C: This case is similar to Case A with the main difference in that it considers to improve the global circuit performance, i.e. gain, bandwidth, speed, etc., using parameters that pertain to the entire circuit, thus are not localized only to a particular topological module, like in Case A. The algorithm explores possible arcs to connect the inputs, outputs, and the internal nodes of a circuit topology.

IV. Experiments

Experiments attempted to characterize the nature of the answers produced by an LLM in response to prompts on analog circuit schematics design, as they are critical in understanding the necessary elaboration algorithms using SKG. Prompt formulation followed the methodology in Fig. 2, and a chain-of-thought, zero-shot approach. The study considered different prompts starting from simpler and moving towards more complex, and identified the LLM limitations in design elaboration. Microsoft's Copilot LLM was used in the experiments, but similar results are expected for any LLM.

1. Changing the verb in a prompt. The simplest prompt parameters are the verbs used to describe the problem description (requirements). For example, the requirement to improve the slew rate (SR) of a CMOS amplifier can be formulated using other similar verbs instead of "improve", like "increase", "maximize", and "enhance". Opposite description are also possible, e.g., decrease the SR of a CMOS amplifier. The answers received for these prompts were very similar, e.g., they included the alternatives to increase bias current, use stronger transistors, minimize load capacitance, and implement a compensation network for stability. The different verbs produced the following changes: other verbs were used to replace "increase", i.e. "boost", "max out", and "maximize", "implement" was replaced with "add", "adjust", "optimize", and "fine-tune", and "stronger" was substituted by "higher transconductance" or "high transconductance". Hence, different expressions of the performance requirement prompt did not change the meaning of the obtained answers.

2. Prompts to obtain the initial topological modules. This experiment studied creating different prompts to obtain the initial features about a circuit schematics. The considered requirements referred to high speed, low voltage, low power OTAs, e.g., the circuits in [13]. According to equation (1), prompts can refer to (i) tradeoffs, like "enumerate the tradeoffs of low voltage, low power OTAs", (ii) constraints, i.e. "enumerate the main design constraints of low voltage, low power OTAs", and (iii) design features, e.g., "describe the features of low voltage, low power OTAs". For case (i), LLM responses included "power consumption Vs. speed and gain, noise due to low supply voltage, high gain is hard, reduced bandwidth, stability requires careful compensation, increased leakage current, which increases power, and increased complexity." These responses are correct but not always explicitly

Authorized licensed use limited to: University of Florida. Downloaded on September 28,2026 at 03:52:25 UTC from IEEE Xplore. Restrictions apply.

presented as tradeoffs, i.e. low voltage Vs. power tradeoff. The speed Vs. power tradeoff is not mentioned, hence, the incomplete response must be addressed through the TPT of SKG. For case (ii), the LLM response included "supply voltage: headroom for device operation, power consumption: current availability, gain: low headroom, noise: low supply, output swing: due to limited supply, stability: requires compensation, thermal management: performance degradation due to thermal issues, parasitic capacitances: more important at low voltage, and process variation: more important at low voltage". Adding details to the prompt, i.e. adding "with supply voltage of 2V" to the prompt, offered additional specificity, like "the low transistor g_m, the impact of the bias currents on power, and leakage currents". Changing the mentioned value from 2V to 1V did not result in additional information. Hence, the LLM does not have enough capabilities to understand the implications of numeric values on the design schematics. For case (iii), e.g., "describe the features of low voltage, low power OTAs", the responses included "high gain - bandwidth product, high open-loop differential gain, high input resistance, low output resistance, low output offset voltage, and high CMRR". SKG must compensate for less insight about bottlenecks.

3. Finding topological modules for specified requirements. This study explored the LLM's capability to identify topological modules for specified tradeoffs, like responding to the prompt "enumerate circuit topologies to tackle the speed power tradeoff of low voltage, low power OTAs". The LLM response included two-stage OTA: differential input and common-source output, folded cascode, class AB amplifiers, current mirror load, gain boosting: bootstrapping, dynamic threshold MOSFETs, switched capacitor circuits, clocked CMOS, through single-phase clock, and body-driven transistors. Adding "with supply voltage of 1V" to the prompt added self-cascoded configuration and subthreshold circuits to the list. Same results were obtained if the value was changed to 2V, which suggests that the LLM cannot react to specific design parameter values.

4. Elaboration of selected topologies. Assuming that the synthesis methodology decided to use class AB amplifiers from the enumerated list, the LLM was prompted to elaborate on the design. The used prompt was "enumerate the type of class AB topologies used in low voltage, low power OTAs." The response included rail-to-rail class AB output stage: maximizes SR, adaptive biasing class AB: nonlinear current mirror, class AB with floating capacitors, class AB with feedback control, and class AB with source follower and common sources stages. Adding "1V supply voltage" to the prompt produced two additional options: "dynamic biasing class AB: biasing scheme to control quiescent current and nonlinear adaptive biasing: nonlinear current mirror to enhance the maximum tail current as a function of the input voltage achieving super class AB behavior". More topological modules were obtained using the prompt "enumerate cascode topologies with gain boosting in low voltage, low power OTAs". The responses included "composite cascode: cascode and transistors to increase gain, split folded cascode, indirect compensation cascode, gain-boosted folded cascode, and body-effect cascode." Additional topology options resulted by elaborating each response option.

5. Insight about topology performance. The study explored the nature of the performance characterization given by the LLM for a topology. For example, the prompt "for a low voltage, low power OTA, enumerate the performance modified by the cascode configuration" offered the following answer: "increased gain, enhanced bandwidth, improved output impedance, reduced parasitic capacitance, better linearity, improved isolation, and increased SR indirectly because of gain and bandwidth". Hence, the LLM did not offer detailed quantitative information to tackle tradeoffs and bottlenecks.

V. Conclusion

This paper presents a novel method to integrate an LLM with a Symbolic Knowledge Graph (SKG) to create optimized analog circuit schematics. The paper argues that LLMs are limited in elaborating to generate a detailed MOSFET-level schematics, and cannot adjust modules based on quantitative performance. An SKG is proposed to compensate for these limitations. Experiments support the stated LLM limitations, which then can serve as starting points of future work to develop the needed SKG features and algorithms.

Acknowledgment

This work relates to Department of Navy award N00014-22-1-2001 issued by the Office of Naval Research. The United States Government has a royalty-free license throughout the world in all copyrightable material contained herein.

References

[1] X. Wu and et al., "A Survey of Human-in-the-loop for Machine Learning", Future Generation Computer Systems, 135, pp.364-381, 2022.
[2] C. Bishop, Deep Learning, Springer, 2024.
[3] G. Gielen and R. Rutenbar, "Computer aided design of analog and mixed-signal integrated circuits", Proc. IEEE, 88, pp. 1825–1852, 2000.
[4] A. Doboli and R. Vemuri, "Exploration-Based High-Level Synthesis of Linear Analog Systems Operating at Low/Medium Frequencie", IEEE Transactions CADICS, 22(11), pp. 1556–1568, 2003.
[5] W. Kruiskamp and D. Leenaerts, "Darwin: Cmos opamp synthesis by means of genetic algorithm", Proc. DAC, 1995, pp. 433–438.
[6] M. Liu, and et al., "VerilogEval: Evaluating Large Language Models for Verilog Code Generation", Proc. ICCAD, 2023.
[7] Z. He and et al., "ChatEDA: A Large Language Model Powered Autonomous Agent for EDA", Proc. MLCAD, 2023.
[8] C.-T. Ho and H. Ren, "Large Language Model (LLM) for Standard Cell Layout Design Optimization", arXiv, 2406.06549, 2024, https://arxiv.org/abs/2406.06549.
[9] C.-C. Chang and et al., "LaMAGIC: Language-Model-based Topology Generation for Analog Integrated Circuits", ICML, 2024.
[10] C. Liu and et al, "LADAC: Large Language Model-driven Auto-Designer for Analog Circuits", techrxiv, 10.36227/techrxiv.170473941.10097233/v1, 2024.
[11] J. Shen and et al., "Atelier: An Automated Analog Circuit Design Framework via Multiple Large Language Model-based Agents", 10.36227/techrxiv.172668168.88938111/v1, 2024.
[12] F. Jiao and et al., "Analog Circuit Design Knowledge Mining: Discovering Topological Similarities and Uncovering Design Reasoning Strategies," IEEE Transactions CADICS, 34(7), pp. 1045–1058, 2015.
[13] H. Lampinen, and O. Vainio, "An optimization approach to designing otas for low-voltage sigma-delta modulators", IEEE Transactions on Instrumentation and Measurement, 50(6), pp.1665–1671, 2001.
[14] X. Liu and et al., "BottomUp Modeling of Design Knowledge Evolution: Application to Circuit Design Community Characterization", IEEE Transactions CSS, 8(3), pp. 689–703, 2021.

Authorized licensed use limited to: University of Florida. Downloaded on September 28,2026 at 03:52:25 UTC from IEEE Xplore. Restrictions apply.